

Compte rendu : Systèmes Linéaires à Grande Dimension (SLGD)

Matthieu Keruzoret, Jasim Sali

13 janvier 2025

Introduction

Dans ce compte rendu, nous allons aborder différentes méthodes itératives de résolution d'un système matriciel de la forme $Ax = b$ où A est de taille $n \times n$ et b et x sont de taille $n \times 1$. Comme première approche pour résoudre le système, nous pouvons faire : $x = A^{-1}b$. Cependant, cela impliquerait de calculer A^{-1} qui a un coût de calcul de l'ordre de $O(n^3)$, ce qui peut être extrêmement fastidieux plus la matrice A est grande. L'approche choisie est que nous allons utiliser une suite $x^{(k)}$ qui converge vers un point fixe x , solution du système d'équations linéaires. L'objectif principal est d'examiner les propriétés fondamentales de ces méthodes, telles que leur convergence, leur efficacité d'application, et voir quelques cas choisis.

Nous commencerons par présenter les bases théoriques nécessaires à la compréhension des méthodes itératives. Puis, nous aborderons la méthode de Jacobi, de Gauss-Seidel et la méthode de surrelaxation successive (SOR), notamment les conditions de convergence et les caractéristiques des méthodes employées. Enfin, nous comparerons ces techniques à travers des exemples pratiques, afin d'illustrer leurs avantages, leurs limitations et leur pertinence dans divers contextes applicatifs.

Ce travail vise ainsi à fournir une vision claire et structurée des outils itératifs disponibles pour résoudre efficacement des systèmes linéaires, tout en soulignant les critères de choix selon les besoins spécifiques du problème à traiter.

Table des matières

Introduction	1
1 Prérequis	3
1.1 Matrice creuse	3
1.2 Préconditionnement pour les méthodes itératives	4
1.3 Cas des matrices à diagonale non dominante	4
2 Méthode de Jacobi	5
2.1 Décomposition de la matrice A	5
2.2 Convergence	5
3 Méthode de Gauss-Seidel	6
3.1 Décomposition de la matrice A	6
3.2 Convergence	6
4 Méthode de surrelaxation successive (SOR)	7
4.1 Décomposition de la matrice A	7
4.2 Convergence	7
4.3 ω optimal	7
5 Comparaison des différentes méthodes	8
5.1 Méthode de Jacobi pour les matrices denses vs Méthode de Jacobi pour les matrices creuses	8
5.2 Méthode de Jacobi vs Méthode de Gauss-Seidel vs Méthode SOR	10
5.3 Equation du mouvement d'un pendule simple	12

Chapitre 1

Prérequis

1.1 Matrice creuse

Par définition, une matrice creuse est une matrice qui comporte un grand nombre de 0. De nombreuses opérations matricielles peuvent être réalisées plus efficacement en exploitant la parcimonie. De plus, seuls les éléments non nuls sont les seuls sauvegardés, ce qui engendre de l'efficacité dans la mémoire, et c'est les seuls qui doivent être pris en compte dans les calculs.

Les matrices creuses sont importantes dans :

- Les simulations à grande échelle : Elles sont essentielles pour résoudre de grands systèmes d'équations survenant dans les applications scientifiques et d'ingénierie.
- L'apprentissage automatique : Elles sont utilisées dans divers algorithmes d'apprentissage automatique, tels que les machines à vecteurs de support et les réseaux neuronaux graphiques.
- L'analyse des données : Elles sont utilisées dans la gestion et analyse efficaces de grands ensembles de données avec des représentations éparses.

Pour illustrer, prenons le cas d'une matrice tridiagonale. Une matrice tridiagonale est une matrice creuse avec des éléments non nuls uniquement sur la diagonale principale, la diagonale au-dessus d'elle (sur-diagonale) et la diagonale en dessous d'elle (sous-diagonale).

Pour une matrice de taille 4, on a :

$$A = \begin{pmatrix} 5 & -1 & 0 & 0 \\ -1 & 5 & -1 & 0 \\ 0 & -1 & 5 & -1 \\ 0 & 0 & -1 & 5 \end{pmatrix}$$

Cette structure apparaît fréquemment dans les méthodes numériques de résolution d'équations différentielles, telles que celles rencontrées dans les méthodes aux différences finies.

Il existe plusieurs formats de stockage pour les matrices creuses.

- Stockage de lignes compressées (CSR) : Il faut stocker les éléments non nuls dans un seul tableau. Puis stocker un tableau de nombres cumulés d'entrées non nulles par ligne. Et enfin, stocker les indices de colonne des éléments non nuls dans un troisième tableau.
- Stockage de colonnes compressées (CSC) : Il est similaire au CSR, mais stocke les éléments et les indices non nuls par colonne.
- Liste de coordonnées (COO) : il stocke les indices de ligne et de colonne de chaque élément non nul avec sa valeur.

1.2 Préconditionnement pour les méthodes itératives

Le preconditionnement est une technique utilisée pour accélérer la convergence des méthodes itératives de résolution de systèmes linéaires de la forme $Ax = b$. L'idée principale est de transformer le système d'origine en un système équivalent mieux conditionné, c'est-à-dire qu'il converge plus rapidement avec les méthodes itératives. Nous introduisons une « matrice de preconditionnement » C (invertible) et transformons le système d'origine comme suit :

$$\begin{aligned} Ax &= b \\ C^{-1}Ax &= C^{-1}b \end{aligned}$$

Nous pouvons illustrer cette méthode avec un exemple. Dans le cas de la méthode de Jacobi, on a $C = D$ où D est la matrice diagonale de A :

$$\begin{aligned} D^{-1}(D - (L + U))x &= D^{-1}b \\ (I - D^{-1}(L + U))x &= D^{-1}b \\ x &= D^{-1}b + D^{-1}(L + U)x \end{aligned}$$

1.3 Cas des matrices à diagonale non dominante

Dans cette section, nous allons nous intéresser aux matrices à diagonale non dominante. Prenons pour exemple cette matrice :

$$A = \begin{pmatrix} 1 & -2 & 0 & \dots & 0 \\ -2 & 1 & -2 & \dots & 0 \\ 0 & -2 & 1 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & -2 \\ 0 & \dots & 0 & -2 & 1 \end{pmatrix}$$

D'après le lemme de Hadamard :

Si $A = (a_{i,j})_{(i,j) \in [1,n]^2}$ alors A est invertible.

(cf : Wikipédia)

Sachant que A n'est pas à diagonale dominante, alors elle n'est pas invertible. Ainsi, on ne peut donc pas utiliser les preconditionneurs pour les différentes méthodes et, par conséquent, on ne peut pas utiliser les trois méthodes.

Chapitre 2

Méthode de Jacobi

2.1 Décomposition de la matrice A

Dans le cas de la méthode de Jacobi, on peut décomposer le système matriciel $Ax = b$ sous la forme : $(D + L + U)x = b$ où :

- D est la matrice diagonale de A
- L est la matrice triangulaire inférieure de A
- U est la matrice triangulaire supérieure de A

Ainsi on peut le système devient :

$$\begin{aligned} Dx &= b - (L + U)x \\ x &= D^{-1}b - D^{-1}(L + U)x \end{aligned}$$

Nous obtenons donc la formule itérative de Jacobi :

$$x^{(k+1)} = D^{-1}b - D^{-1}(L + U)x^{(k)}$$

où :

- $x^{(k)}$ est l'actuelle approximation de la solution vectorielle à l'itération k
- $x^{(k+1)}$ est l'approximation mise à jour à l'itération $k+1$

Et chaque composante de la matrice x s'écrit :

$$x_i^{(k+1)} = \frac{1}{a_{ii}}(b_i - \sum_{j \neq i} a_{ij}x_j^{(k)}) \text{ pour } i = 1, 2, \dots, n$$

2.2 Convergence

Une condition nécessaire et suffisante pour la convergence de la méthode de Jacobi est que le rayon spectral de la matrice d'itération $T = D^{-1}(L + U)$ doit être inférieur à 1. On peut l'écrire ainsi :

$$\rho(T) < 1 \text{ où } \rho \text{ désigne le rayon spectral}$$

Le rayon spectral est défini comme étant, en valeur absolue, la valeur propre la plus grande.

Une condition suffisante pour la convergence de la méthode de Jacobi est que cette dernière converge si la matrice A est à diagonale dominante. On dit que A est à diagonale dominante si :

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}| \text{ pour } i=1, 2, \dots, n$$

Cela signifie que la valeur absolue de chaque élément des diagonales doit être plus grand que la somme des valeurs absolues de chaque autre élément de sa ligne.

Chapitre 3

Méthode de Gauss-Seidel

3.1 Décomposition de la matrice A

Dans le cas de la méthode de Gauss-Seidel, nous pouvons utiliser comme préconditionneur du système matricielle $Ax = b$: $C = (L + D)$ ou $C = (U + D)$, où :

- D est la matrice diagonale de A
- $-L$ est la matrice triangulaire inférieure de A
- $-U$ est la matrice triangulaire supérieure de A

Ainsi, le système devient dans le cas où nous utilisons le préconditionneur $C = (L + D)$:

$$\begin{aligned}(L + D)^{-1}(D + U + L)x &= (L + D)^{-1}b \\ (I - (L + D)^{-1}U)x &= (L + D)^{-1}b \\ x &= (L + D)^{-1}b + (L + D)^{-1}Ux\end{aligned}$$

Nous obtenons donc la formule itérative de Gauss-Seidel pour chaque composante de la matrice x :

$$x_i^{(k+1)} = \frac{1}{a_{ii}}(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)}) \text{ pour } i = 1, 2, \dots, n$$

3.2 Convergence

Une condition nécessaire et suffisante pour la convergence de la méthode de Gauss-Seidel est que le rayon spectral de la matrice d'itération $T = (L + D)^{-1}U$ doit être inférieur à 1.

Chapitre 4

Méthode de surrelaxation successive (SOR)

4.1 Décomposition de la matrice A

Dans le cas de la méthode SOR, nous pouvons utiliser comme préconditionneur du système matricielle $Ax = b$, avec le paramètre de relaxation ω ($0 < \omega < 2$) : $C = \frac{1}{\omega}(D + \omega L)$ Ainsi, le système devient :

$$(D + \omega L)x = \omega b - (\omega U + (\omega - 1)D)x$$

Nous obtenons donc la formule itérative de SOR pour chaque composante de la matrice x :

$$s_i^{(k)} = \frac{1}{a_{ii}}(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)}) \text{ pour } i = 1, 2, \dots, n$$
$$x_i^{(k+1)} = \omega s_i^{(k)} + (1 - \omega)x_i^{(k)}$$

4.2 Convergence

Une condition nécessaire et suffisante pour la convergence de la méthode SOR est que le rayon spectral de la matrice d'itération $T = (D + \omega L^{-1})((\omega - 1)D + \omega U)$ doit être inférieur à 1.

4.3 ω optimal

La recherche d'un ω optimal est motivée par le fait de vouloir une méthode SOR toujours convergente. En effectuant quelques recherches, on a trouvé que pour avoir un ω optimal, on devait utiliser la formule suivante :

$$\omega_{optimal} = \frac{2}{1 + (\sqrt{1 - |\rho(T_{jacobi})|^2})}$$

où T_{jacobi} est la matrice d'itération de la méthode de Jacobi.

(cf : Institut de Mathématiques de Marseille)

Chapitre 5

Comparaison des différentes méthodes

5.1 Méthode de Jacobi pour les matrices denses vs Méthode de Jacobi pour les matrices creuses

Dans cette section, nous allons utiliser la matrice tridiagonale suivante :

$$\begin{pmatrix} 5 & -1 & 0 & \dots & 0 \\ -1 & 5 & -1 & \dots & 0 \\ 0 & -1 & 5 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & -1 \\ 0 & \dots & 0 & -1 & 5 \end{pmatrix}$$

Pour une matrice de taille $n = 110$, nous avons :

```
n    Dense Time (s)    Sparse Time (s)
110      0.0024709701538085938      0.00118494033813
```

FIGURE 5.1 – Comparaison Jacobi dense et creuse pour $n = 110$

Pour une matrice de taille $n = 1000$, nous avons :

```
n    Dense Time (s)    Sparse Time (s)
1000    0.0356137752532959      0.001133918762207031
```

FIGURE 5.2 – Comparaison Jacobi dense et creuse pour $n = 1000$

Pour une matrice de taille $n = 10000$, nous avons :

```
n    Dense Time (s)    Sparse Time (s)
10000    1.4142415523529053      0.001697063446044921
```

FIGURE 5.3 – Comparaison Jacobi dense et creuse pour $n = 10000$

On remarque que pour toutes les valeurs de n testées, la méthode de Jacobi pour les matrices creuses est plus rapide.

Cependant, durant nos tests, nous avons remarqués un petit couac.

En effet, en utilisant la méthode de Jacobi pour les matrices denses avec une matrice de taille $n=110$ définie comme ceci :

$$\begin{pmatrix} 5 & -1 & \dots & -1 \\ -1 & 10 & \ddots & \vdots \\ \vdots & \ddots & \ddots & -1 \\ -1 & \dots & -1 & 5(n+1) \end{pmatrix}$$

nous avons :

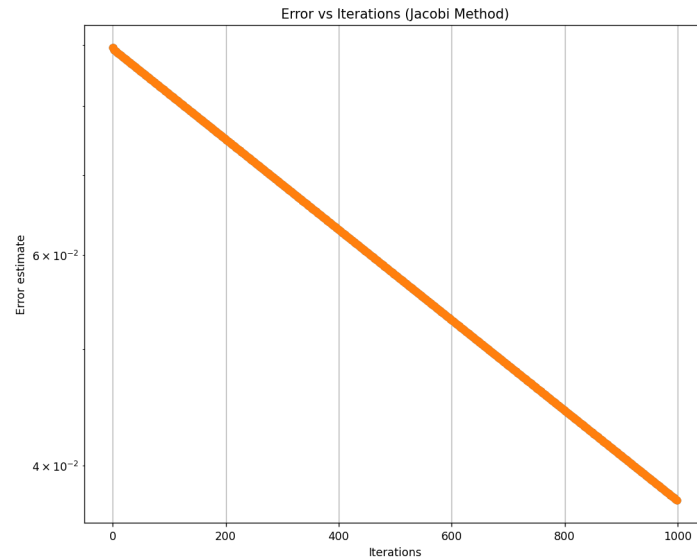


FIGURE 5.4 – Jacobi dense pour $n = 110$

La méthode de Jacobi converge. Le rayon spectral est égal à 0.99913. Et avec une matrice de taille $n=111$, nous avons :

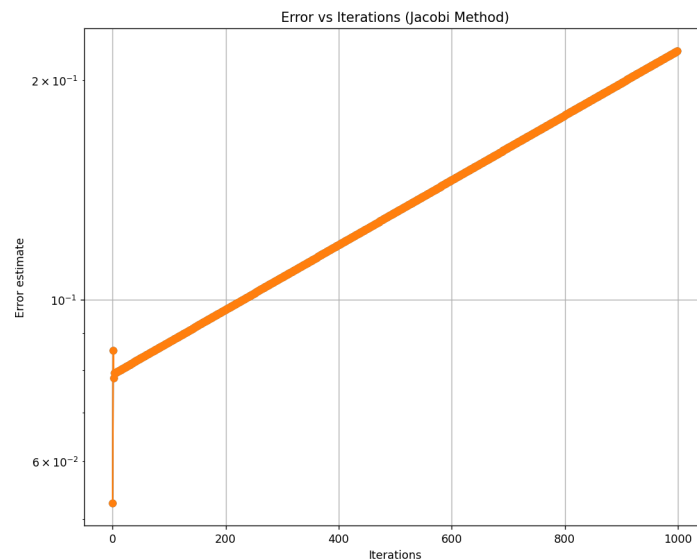


FIGURE 5.5 – Jacobi dense pour $n = 111$

La méthode de Jacobi diverge. Le rayon spectral est égal à 1.00102. Pour tout $n > 110$, l'erreur ne va faire qu'augmenter.

5.2 Méthode de Jacobi vs Méthode de Gauss-Seidel vs Méthode SOR

Dans cette section, nous allons utiliser l'opérateur laplacien aux différences finies du 2e ordre en dimension 1 pour comparer les temps de calcul des différentes méthodes et pour comparer leur convergence.

On définit l'opérateur laplacien aux différences finies du 2e ordre en dimension 1 comme étant :

$$\frac{\partial^2 x}{\partial x^2} = g(x)$$

$$x^{(2)} \approx \frac{1}{h^2}(2f_i - f_{i+1} - f_{i-1}) \text{ où } h \text{ est le pas qui est défini par : } h = \frac{1}{n+1}$$

On peut écrire cela sous la forme matricielle suivante :

$$A = \frac{1}{h^2} \begin{pmatrix} 2 & -1 & 0 & \dots & 0 \\ -1 & 2 & -1 & \dots & 0 \\ 0 & -1 & 2 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & -1 \\ 0 & \dots & 0 & -1 & 2 \end{pmatrix}$$

Pour une matrice de taille $n = 25$, nous avons :

```
n    jac Time (s)    gs Time (s) sor Time (s)
25   0.011094093322753906   2.0920565128326416   0.20
50032615661621
```

FIGURE 5.6 – Comparaison des 3 méthodes pour une matrice creuse de taille $n=25$

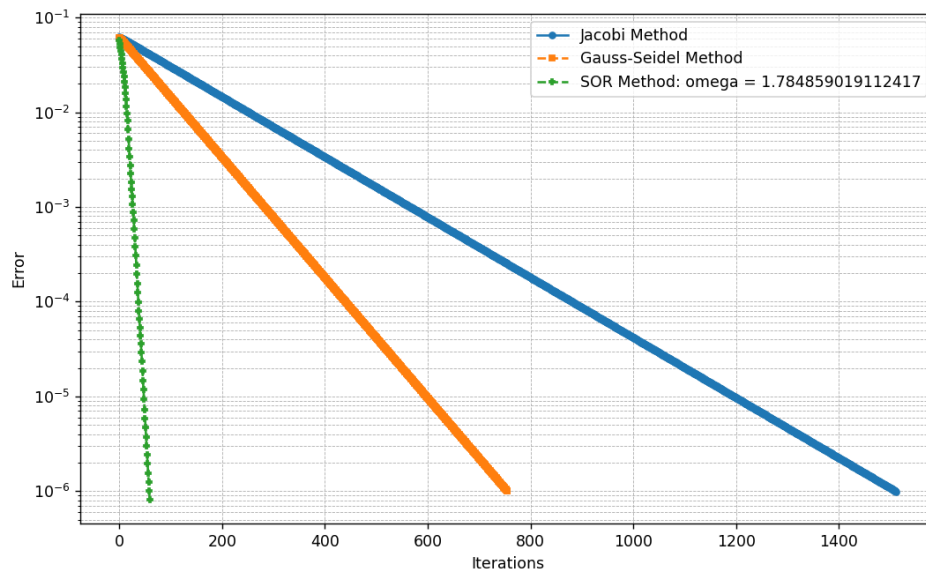


FIGURE 5.7 – Comparaison des 3 méthodes pour une matrice creuse de taille $n=25$

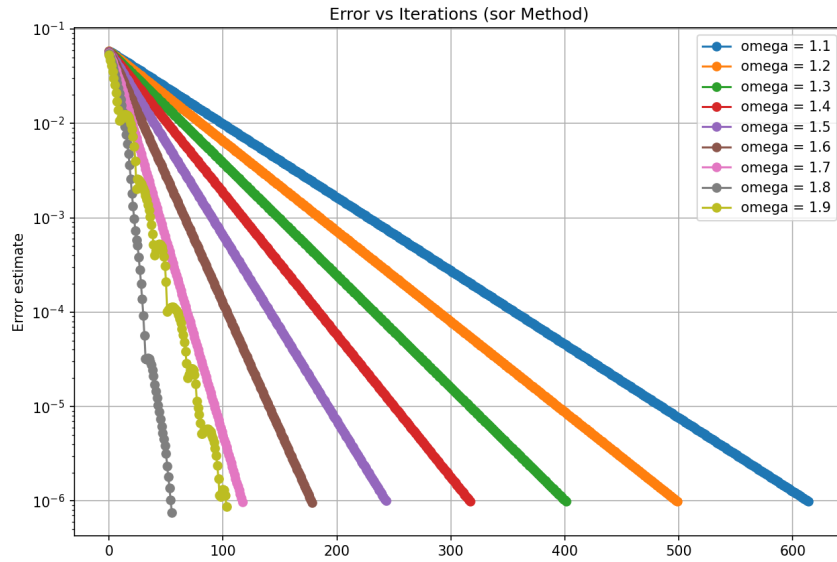


FIGURE 5.8 – Comparaison des ω pour la méthode SOR avec une matrice creuse de taille $n=25$

On remarque dans la figure 5.6 que la méthode de Jacobi a un temps de calcul plus rapide que les deux autres méthodes. Cependant, la figure 5.7 nous montre que la méthode SOR converge beaucoup plus vite que les deux autres méthodes.

La figure 5.8 nous informe que le ω optimal est compris entre : $1.7 < \omega_{optimal} < 1.9$, ce qui est conforme avec la valeur utilisée dans la figure 5.6 et 5.7.

Pour une matrice de taille $n = 110$, nous avons :

n	jac Time (s)	gs Time (s)	sor Time (s)
110	0.08353781700134277	96.29596590995789	2.4094862937927246

FIGURE 5.9 – Comparaison des 3 méthodes pour une matrice creuse de taille $n=110$

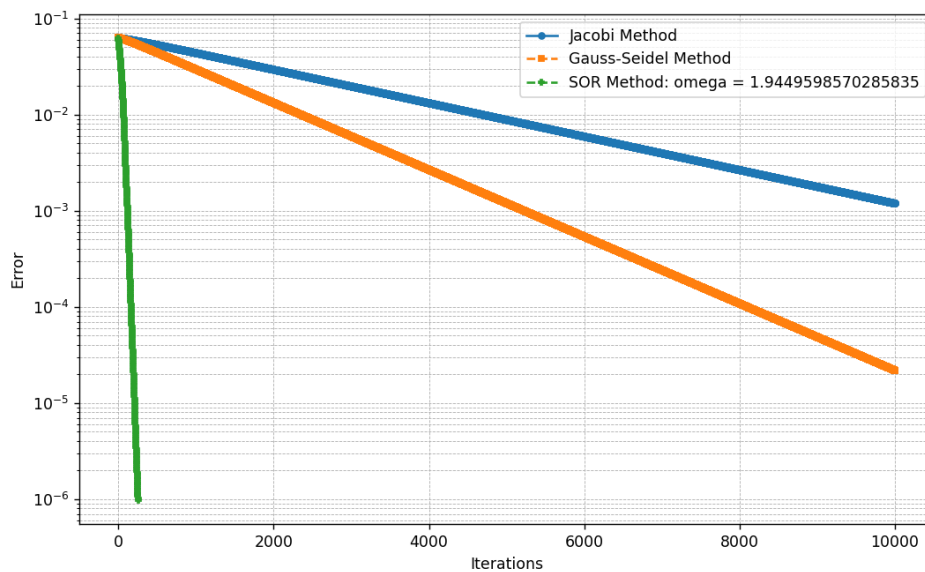


FIGURE 5.10 – Comparaison des 3 méthodes pour une matrice creuse de taille $n=110$

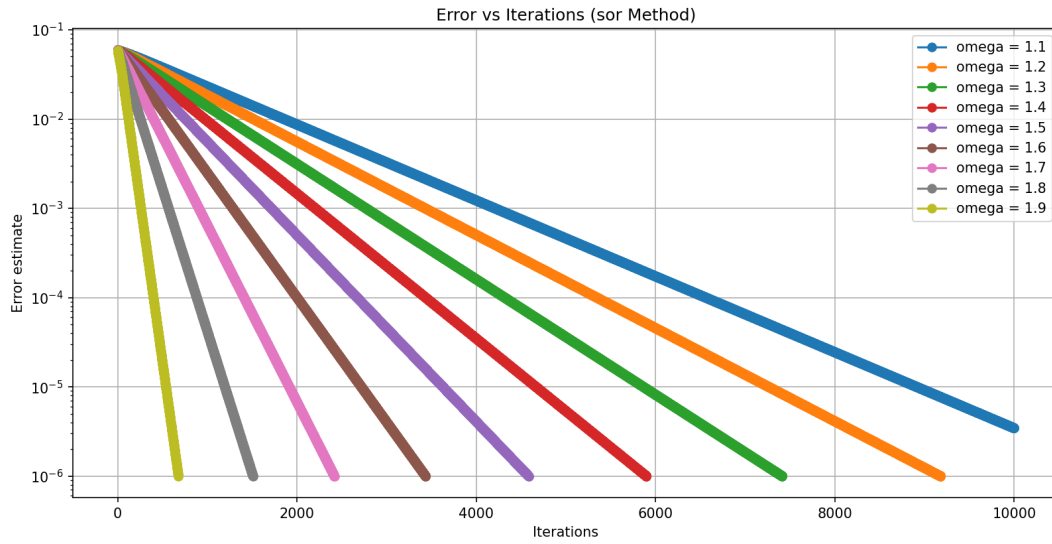


FIGURE 5.11 – Comparaison des ω pour la méthode SOR avec une matrice creuse de taille $n=110$

On remarque dans la figure 5.9 que la méthode de Jacobi a un temps de calcul plus rapide que les deux autres méthodes. De plus, la méthode de Gauss-Seidel a un temps de calcul très élevé. Cependant, la figure 5.10 nous montre que la méthode SOR converge beaucoup plus vite que les deux autres méthodes.

La figure 5.11 nous informe que le ω optimal est compris entre : $1.8 < \omega_{optimal} < 2$, ce qui est conforme avec la valeur utilisée dans la figure 5.9 et 5.10.

5.3 Equation du mouvement d'un pendule simple

Dans cette section, nous allons nous intéresser à l'équation du mouvement d'un pendule simple.

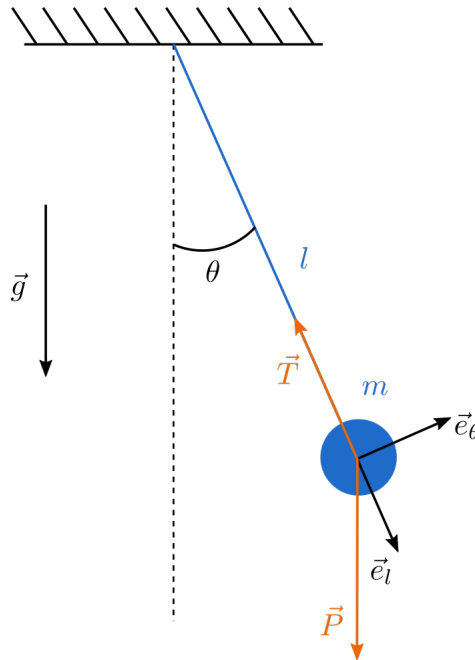


FIGURE 5.12 – Schéma du pendule simple représentant les forces en jeu et les vecteurs unitaires en coordonnées polaires

Elle est définie comme suit :

$$\ddot{\theta} + \omega_0^2 \sin(\theta) = 0$$

Or si θ est petit, alors : $\sin(\theta) = \theta$

On peut donc écrire que : $\ddot{\theta} + \omega_0^2 \theta = 0$

où :

- θ est l'angle formé par le pendule simple avec la verticale descendante.
- $\ddot{\theta}$ est l'accélération angulaire du pendule.
- ω_0 désigne la pulsation propre du pendule. $\omega_0^2 = \frac{g}{l}$ où g est l'accélération due à la pesanteur et l désigne la longueur du fil.

De la même manière que la matrice construite dans la section 5.2, on peut écrire l'équation ci-dessus de la manière suivante :

$$A = \frac{1}{h^2} \begin{pmatrix} 2 + (h\omega_0)^2 & -1 & 0 & \dots & 0 \\ -1 & 2 + (h\omega_0)^2 & -1 & \dots & 0 \\ 0 & -1 & 2 + (h\omega_0)^2 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & -1 \\ 0 & \dots & 0 & -1 & 2 + (h\omega_0)^2 \end{pmatrix}$$

où h est le pas qui est défini par : $h = \frac{1}{n+1}$

Pour une matrice de taille $n = 25$, on obtient :

```
n   jac Time (s)   gs Time (s) sor Time (s)
25  0.008315086364746094  0.2236347198486328  0.047974586486816406
```

FIGURE 5.13 – Comparaison des 3 méthodes pour la matrice creuse de taille $n=25$

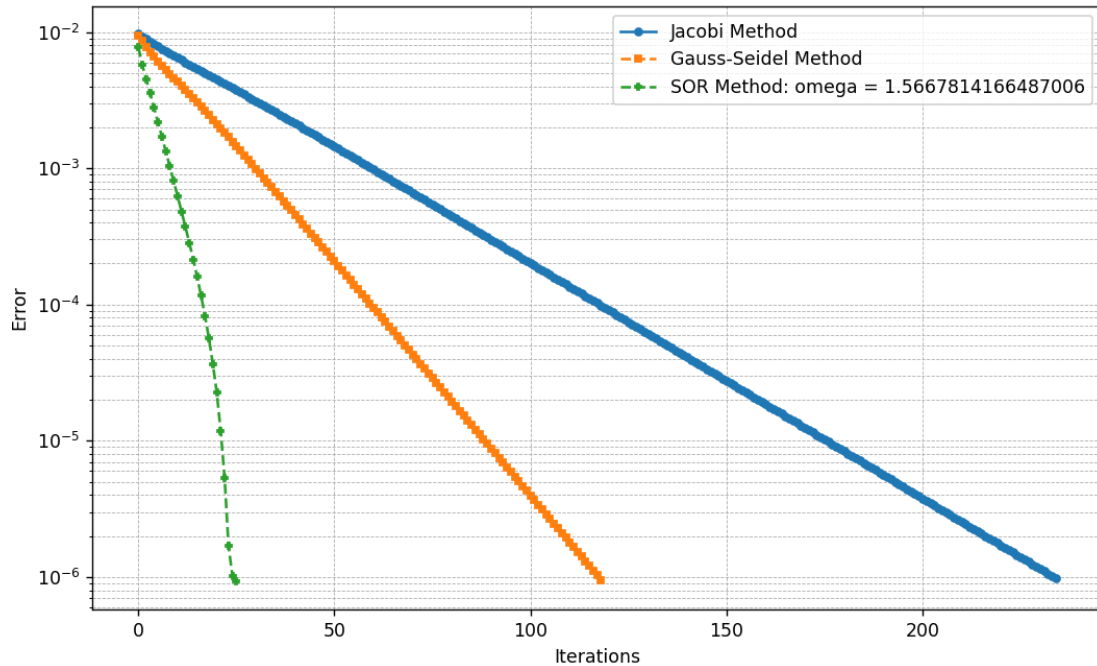


FIGURE 5.14 – Comparaison des 3 méthodes pour la matrice creuse de taille $n=25$

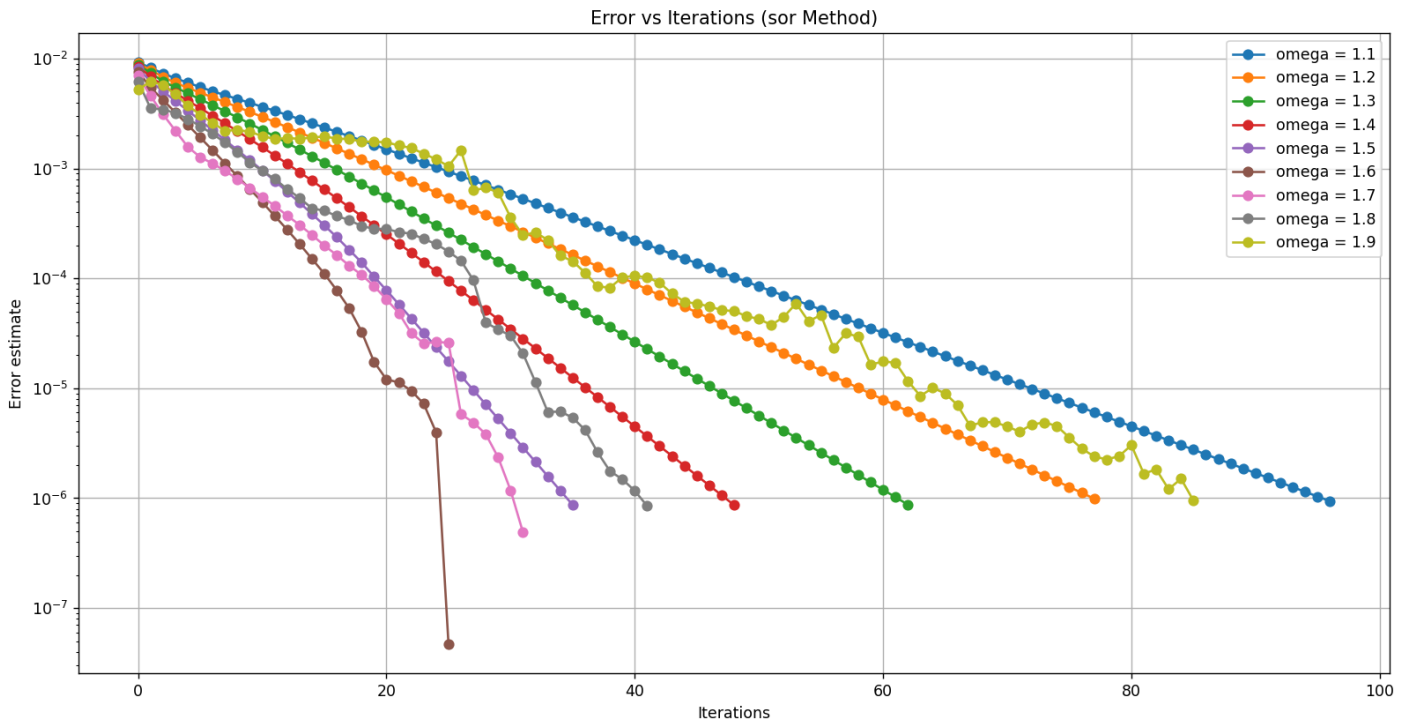


FIGURE 5.15 – Comparaison des ω pour la méthode SOR avec la matrice creuse de taille $n=25$

On remarque dans la figure 5.13 que la méthode de Jacobi a un temps de calcul plus rapide que les deux autres méthodes. De plus, la méthode de Gauss-Seidel a un temps de calcul assez élevé comparé à la méthode de Jacobi et SOR. Cependant, la figure 5.14 nous montre que la méthode SOR converge beaucoup plus vite que les deux autres méthodes.

La figure 5.15 nous informe que le ω optimal est compris entre : $1.5 < \omega_{optimal} < 1.7$, ce qui est conforme avec la valeur utilisée dans la figure 5.13 et 5.14.